

Trabalho:Previsão de Séries Temporais utilizando Redes LSTM

sistemas inteligentes – UFSM

Alunos:Diogo Rocha e Gabriel Quadros

1. Introdução

A previsão de séries temporais é uma área de estudo da Inteligência Artificial e do Aprendizado de Máquina que busca estimar valores futuros com base em observações realizadas ao longo do tempo. Esse tipo de problema está presente em diversas áreas do conhecimento, como economia, engenharia, meteorologia, medicina, logística e mercado financeiro. A capacidade de prever tendências futuras auxilia empresas e organizações na tomada de decisões, permitindo melhor planejamento, redução de custos e otimização de recursos.

Entre as diversas técnicas utilizadas para previsão de séries temporais, as Redes Neurais Artificiais têm se destacado devido à sua capacidade de aprender padrões complexos presentes nos dados. Em especial, as Redes Neurais Recorrentes (Recurrent Neural Networks – RNN) foram desenvolvidas para trabalhar com dados sequenciais, permitindo que informações de estados anteriores influenciem as previsões futuras.

Apesar das vantagens apresentadas pelas RNNs tradicionais, essas arquiteturas possuem limitações relacionadas ao aprendizado de dependências de longo prazo. Durante o treinamento, problemas como o desaparecimento (Vanishing Gradient) e a explosão do gradiente (Exploding Gradient) dificultam o armazenamento de informações importantes provenientes de observações mais antigas da sequência, reduzindo a capacidade preditiva do modelo.

Como solução para essas limitações, foram propostas as redes Long Short-Term Memory (LSTM), uma arquitetura especializada de Redes Neurais Recorrentes capaz de armazenar

informações relevantes durante períodos mais longos. As redes LSTM utilizam mecanismos internos conhecidos como portas (gates), responsáveis por controlar quais informações devem ser esquecidas, quais devem ser armazenadas e quais serão utilizadas para produzir a saída da rede.

Neste trabalho foi desenvolvido um sistema para previsão de séries temporais utilizando redes LSTM implementadas em Python com a biblioteca TensorFlow/Keras. O projeto foi estruturado seguindo os princípios da Programação Orientada a Objetos (POO) e do Clean Code, organizando o sistema em classes independentes responsáveis pelo carregamento dos dados, normalização, geração das sequências temporais, construção da rede neural, treinamento, realização das previsões, avaliação dos resultados e geração de gráficos.

Como conjunto de dados foi utilizado o dataset Airline Passengers, composto pelo número mensal de passageiros transportados por companhias aéreas internacionais entre os anos de 1949 e 1960. Esse conjunto é amplamente utilizado na literatura científica e em aplicações educacionais relacionadas à previsão de séries temporais, tornando-se uma referência para avaliação de modelos baseados em aprendizado profundo.

Após o treinamento da rede LSTM, foram realizadas previsões sobre o conjunto de teste e calculadas métricas quantitativas de desempenho, incluindo o Erro Quadrático Médio (Mean Squared Error – MSE), a Raiz do Erro Quadrático Médio (Root Mean Squared Error – RMSE) e o Erro Absoluto Médio (Mean Absolute Error – MAE). Além disso, foram produzidos gráficos comparando os valores reais e previstos, permitindo uma análise visual da capacidade de generalização do modelo.

Ao final do trabalho, busca-se compreender como as redes LSTM podem ser aplicadas em problemas envolvendo dados temporais, demonstrando suas vantagens em relação às Redes Neurais Recorrentes tradicionais e evidenciando sua importância em aplicações modernas de Inteligência Artificial voltadas para previsão de séries temporais.

2. Fundamentação Teórica

2.1 Redes Neurais Artificiais

As Redes Neurais Artificiais (RNAs) são modelos computacionais inspirados no funcionamento do cérebro humano, sendo compostas por unidades de processamento denominadas neurônios artificiais. Esses neurônios são organizados em camadas interligadas, permitindo que a rede aprenda relações entre entradas e saídas por meio de exemplos fornecidos durante o treinamento.

O processo de aprendizado ocorre através do ajuste dos pesos das conexões entre os neurônios. Inicialmente, esses pesos são definidos de forma aleatória e, durante o treinamento, são atualizados utilizando algoritmos de otimização, como o Gradiente Descendente (Gradient Descent) e o algoritmo Adam (Adaptive Moment Estimation). O

objetivo desse processo é minimizar uma função de perda (Loss Function), responsável por medir o erro entre os valores previstos pelo modelo e os valores reais presentes no conjunto de dados.

As Redes Neurais Artificiais são amplamente utilizadas em diversas aplicações, como reconhecimento de imagens, processamento de linguagem natural, classificação de padrões, sistemas de recomendação e previsão de séries temporais. Entretanto, as arquiteturas tradicionais, conhecidas como Feedforward Neural Networks, apresentam limitações quando os dados possuem dependência temporal, pois consideram cada entrada de forma independente.

2.2 Redes Neurais Recorrentes (RNN)

As Redes Neurais Recorrentes (Recurrent Neural Networks – RNN) foram desenvolvidas para lidar com dados sequenciais, permitindo que informações provenientes de etapas anteriores influenciem as previsões realizadas nas etapas seguintes. Diferentemente das redes neurais convencionais, as RNN possuem conexões recorrentes, formando um mecanismo de memória capaz de armazenar informações temporárias durante o processamento da sequência.

Em uma RNN, cada elemento da sequência é processado juntamente com um estado oculto (Hidden State), responsável por transportar informações das observações anteriores. Dessa forma, o modelo consegue capturar relações temporais presentes em séries históricas, tornando-se adequado para aplicações como reconhecimento de fala, tradução automática, processamento de linguagem natural, previsão financeira e análise de séries temporais.

Matematicamente, o estado oculto pode ser representado por:

$$h_t = f(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t + b)$$

onde:

- h_t representa o estado oculto atual;
- h_{t-1} representa o estado oculto anterior;
- x_t corresponde à entrada no instante atual;
- W representam os pesos aprendidos durante o treinamento;
- b corresponde ao vetor de bias;
- f representa a função de ativação, normalmente a tangente hiperbólica ($\tanh$).

Esse mecanismo permite que a rede utilize informações passadas durante a geração das previsões futuras.

2.3 Limitações das Redes Neurais Recorrentes

Embora as Redes Neurais Recorrentes tenham representado um avanço significativo em problemas envolvendo dados sequenciais, elas apresentam limitações importantes relacionadas ao aprendizado de dependências de longo prazo.

Durante o treinamento, o algoritmo Backpropagation Through Time (BPTT) propaga os gradientes ao longo de toda a sequência temporal. Em sequências muito longas, esses gradientes podem tornar-se extremamente pequenos ou excessivamente grandes, originando dois problemas conhecidos como Vanishing Gradient e Exploding Gradient.

O Vanishing Gradient ocorre quando os gradientes diminuem progressivamente até valores próximos de zero. Como consequência, os pesos associados às informações mais antigas praticamente deixam de ser atualizados, fazendo com que a rede "esqueça" acontecimentos importantes ocorridos no início da sequência.

Já o Exploding Gradient ocorre quando os gradientes crescem excessivamente durante o treinamento, ocasionando atualizações muito grandes nos pesos da rede. Esse comportamento pode causar instabilidade numérica e impedir a convergência do modelo.

Esses problemas reduzem significativamente a capacidade das RNN tradicionais de aprender dependências temporais de longo prazo, principalmente em séries históricas extensas. Como solução para essas limitações, foi proposta a arquitetura Long Short-Term Memory (LSTM), capaz de controlar explicitamente o fluxo de informações por meio de mecanismos internos denominados portas (gates), permitindo preservar informações relevantes durante períodos muito maiores do que aqueles alcançados pelas RNN convencionais.

2.4 Redes Long Short-Term Memory (LSTM)

As redes Long Short-Term Memory (LSTM) constituem uma arquitetura especializada de Redes Neurais Recorrentes proposta por Hochreiter e Schmidhuber em 1997 com o objetivo de solucionar as limitações das RNN tradicionais relacionadas ao aprendizado de dependências de longo prazo.

A principal característica das redes LSTM é a utilização de uma célula de memória (Cell State), responsável por armazenar informações relevantes ao longo da sequência temporal. Diferentemente das RNN convencionais, nas quais todas as informações são propagadas diretamente pelo estado oculto, as LSTM possuem mecanismos internos capazes de controlar o fluxo de dados que entram, permanecem ou saem da memória.

Esse controle é realizado por três estruturas denominadas portas (gates):

- Forget Gate (porta de esquecimento);
- Input Gate (porta de entrada);
- Output Gate (porta de saída).

Cada porta é implementada por uma camada neural que utiliza a função de ativação sigmoide. Como essa função produz valores entre 0 e 1, cada porta pode decidir quanto de determinada informação deve ser mantido ou descartado.

Além das portas, a arquitetura LSTM utiliza uma função de ativação tangente hiperbólica (tanh), responsável por gerar novos valores candidatos a serem armazenados na memória.

A combinação desses mecanismos permite que a rede memorize informações importantes durante longos períodos, tornando as LSTM extremamente eficientes em aplicações como previsão de séries temporais, reconhecimento de voz, tradução automática, processamento de linguagem natural, previsão financeira, detecção de anomalias e análise de dados sequenciais.

2.5 Forget Gate

A Forget Gate, ou porta de esquecimento, é a primeira etapa do processamento realizado por uma célula LSTM. Sua função consiste em decidir quais informações armazenadas anteriormente continuarão sendo relevantes e quais deverão ser removidas da memória.

Essa decisão é tomada considerando duas informações:

- a entrada atual da sequência (x_t);
- o estado oculto anterior (h_{t-1}).

Esses valores são combinados e processados por uma camada neural com função de ativação sigmoide.

A equação da Forget Gate é:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

onde:

- f_t representa o vetor de esquecimento;
- σ corresponde à função sigmoide;
- W_f representa os pesos aprendidos pela rede;
- b_f corresponde ao vetor de bias.

Como a função sigmoide produz valores entre 0 e 1, cada posição da memória recebe um peso indicando sua importância. Valores próximos de 1 significam que aquela informação deve permanecer armazenada, enquanto valores próximos de 0 indicam que ela deve ser descartada.

Esse mecanismo evita que informações irrelevantes permaneçam ocupando espaço na memória da rede durante o processamento da sequência.

2.6 Input Gate

Após decidir quais informações devem ser esquecidas, a rede utiliza a Input Gate para determinar quais novas informações serão incorporadas ao estado da célula.

Inicialmente, uma camada sigmoide identifica quais posições da memória poderão receber novos dados. Em seguida, uma camada utilizando a função tangente hiperbólica ($\tanh$) produz um conjunto de valores candidatos a serem armazenados.

As equações utilizadas são:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

Posteriormente, o estado da memória é atualizado pela expressão:

$$C_t = f_t \times C_{t-1} + i_t \times \tilde{C}_t$$

onde:

- C_t representa o estado atual da memória;
- C_{t-1} representa o estado anterior;
- i_t representa a porta de entrada;
- $\tilde{C}_t$ corresponde às novas informações candidatas.

Esse processo garante que apenas informações consideradas importantes sejam adicionadas à memória da rede.

2.7 Output Gate

A última etapa da arquitetura LSTM é realizada pela Output Gate, responsável por definir quais informações armazenadas na memória serão utilizadas como saída da célula no instante atual.

Inicialmente, a porta calcula um vetor utilizando a função sigmoide:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Em seguida, o novo estado oculto é calculado por:

$$h_t = o_t \times \tanh(C_t)$$

onde:

- h_t representa a saída da célula;
- C_t corresponde ao estado atualizado da memória;
- o_t representa a porta de saída.

A Output Gate garante que apenas as informações mais relevantes sejam propagadas para a próxima célula da sequência, contribuindo para que o modelo produza previsões mais precisas.

2.8 Funcionamento Geral da Arquitetura LSTM

O funcionamento completo de uma célula LSTM pode ser resumido em quatro etapas principais:

1. A Forget Gate analisa a memória existente e remove informações consideradas irrelevantes.
2. A Input Gate seleciona novas informações que deverão ser armazenadas na memória da célula.
3. O estado da memória é atualizado combinando as informações preservadas com os novos dados aprendidos.
4. A Output Gate define quais informações presentes na memória serão utilizadas para produzir a saída da rede e alimentar a próxima célula da sequência.

Essa arquitetura permite que as redes LSTM preservem informações relevantes durante longos períodos, solucionando os problemas de desaparecimento e explosão do gradiente encontrados nas Redes Neurais Recorrentes tradicionais. Por esse motivo, as LSTM tornaram-se uma das arquiteturas mais utilizadas em aplicações envolvendo dados sequenciais e séries temporais, apresentando elevado desempenho em tarefas de previsão de valores futuros.

3. Descrição do Dataset

Para o desenvolvimento deste trabalho foi utilizado o conjunto de dados **Airline Passengers**, um dos datasets mais utilizados na literatura para estudos envolvendo previsão de séries temporais e aprendizado profundo. Esse conjunto de dados registra a quantidade mensal de passageiros transportados por companhias aéreas internacionais entre os anos de 1949 e 1960.

O dataset possui um total de **144 registros**, correspondentes aos doze meses de cada ano do período analisado. Cada registro contém duas informações: a data da observação (mês e ano) e o número total de passageiros transportados naquele mês.

A principal característica desse conjunto de dados é apresentar um comportamento não estacionário, com tendência de crescimento ao longo dos anos e sazonalidade bem definida. Observa-se que, em determinados meses do ano, ocorre aumento recorrente na quantidade de passageiros, especialmente durante períodos de férias, enquanto em outros meses há redução na demanda. Essa combinação de tendência e sazonalidade torna o dataset adequado para avaliar modelos capazes de aprender padrões temporais complexos.

Neste trabalho, a variável utilizada para treinamento da rede neural foi exclusivamente o número de passageiros transportados mensalmente. A coluna contendo as datas foi utilizada apenas para preservar a ordem cronológica dos registros, enquanto a coluna de passageiros constituiu a série temporal utilizada pelo modelo.

Antes do treinamento da rede LSTM, os dados passaram por uma etapa de pré-processamento. Inicialmente, os registros foram carregados a partir do arquivo CSV. Em seguida, os valores numéricos foram normalizados utilizando o algoritmo MinMaxScaler, que transforma todos os valores para o intervalo entre 0 e 1. Esse procedimento facilita o processo de treinamento da rede neural, reduzindo diferenças de escala entre os valores e contribuindo para uma convergência mais eficiente do algoritmo de otimização.

Após a normalização, foram construídas sequências temporais utilizando uma janela deslizante (sliding window). Neste projeto, cada amostra foi composta pelos valores correspondentes aos doze meses anteriores, sendo utilizada para prever o valor do mês seguinte. Dessa forma, a rede neural recebe como entrada uma sequência de observações históricas e aprende a estimar o próximo valor da série temporal.

Por fim, o conjunto de dados foi dividido em subconjuntos de treinamento e teste. O conjunto de treinamento foi utilizado para ajustar os pesos da rede neural, enquanto o conjunto de teste foi reservado para avaliar a capacidade de generalização do modelo em dados que não foram utilizados durante o processo de aprendizagem.

4. Metodologia

O desenvolvimento do sistema foi realizado utilizando a linguagem Python e a biblioteca TensorFlow/Keras para implementação da rede neural LSTM. Além disso, foram utilizadas bibliotecas auxiliares como Pandas para manipulação dos dados, NumPy para operações numéricas, Matplotlib para geração de gráficos e Scikit-learn para normalização dos dados e cálculo das métricas de avaliação.

A arquitetura do software foi desenvolvida seguindo os princípios da Programação Orientada a Objetos (POO) e do Clean Code, organizando cada responsabilidade em uma classe específica. Essa abordagem torna o sistema mais modular, reutilizável e de fácil manutenção.

O funcionamento do sistema pode ser dividido nas seguintes etapas.

4.1 Carregamento dos Dados

Inicialmente, o arquivo CSV contendo o dataset foi carregado utilizando a biblioteca Pandas. Durante essa etapa, foi realizada a validação da existência do arquivo e da estrutura esperada das colunas, garantindo que os dados pudessem ser utilizados corretamente nas etapas seguintes.

4.2 Normalização

Após o carregamento dos dados, os valores numéricos foram normalizados utilizando o algoritmo MinMaxScaler.

A normalização transforma todos os valores da série temporal para o intervalo entre 0 e 1, reduzindo diferenças de escala e facilitando o processo de treinamento da rede neural.

Matematicamente, a normalização pode ser representada pela seguinte expressão:

$$x' = (x - x_{\min}) / (x_{\max} - x_{\min})$$

onde:

- x representa o valor original;
- $x_{\min}$ corresponde ao menor valor da série;
- $x_{\max}$ corresponde ao maior valor observado;
- x' representa o valor normalizado.

4.3 Geração das Sequências Temporais

Como as redes LSTM trabalham com dados sequenciais, foi necessário transformar a série temporal em um conjunto de amostras compostas por janelas deslizantes.

Neste projeto foi utilizada uma janela contendo doze observações consecutivas. Cada sequência formada pelos doze meses anteriores foi utilizada como entrada da rede, enquanto o décimo terceiro mês correspondeu ao valor esperado durante o treinamento.

Essa estratégia permite que o modelo aprenda relações temporais presentes na série histórica.

4.4 Divisão entre Treinamento e Teste

Após a geração das sequências, os dados foram divididos em dois subconjuntos.

O conjunto de treinamento foi utilizado para ajustar os pesos da rede neural durante o processo de aprendizagem.

O conjunto de teste permaneceu separado durante o treinamento, sendo utilizado apenas para avaliar o desempenho do modelo após sua conclusão.

Essa separação é fundamental para verificar a capacidade de generalização da rede neural.

4.5 Construção da Rede LSTM

A arquitetura implementada é composta por duas camadas LSTM, responsáveis pela extração das características temporais presentes na série de dados.

Entre as camadas LSTM foram utilizadas camadas de Dropout, responsáveis por reduzir o risco de overfitting durante o treinamento.

Ao final da arquitetura foram adicionadas camadas densamente conectadas (Dense), sendo a última composta por apenas um neurônio, responsável por produzir a previsão do próximo valor da série temporal.

O treinamento foi realizado utilizando o algoritmo de otimização Adam e a função de perda Mean Squared Error (MSE), amplamente utilizada em problemas de regressão.

4.6 Treinamento da Rede

Durante o treinamento foram utilizados mecanismos adicionais para melhorar o desempenho do modelo.

O EarlyStopping interrompe automaticamente o treinamento quando não há melhora significativa na função de perda após determinado número de épocas.

O ReduceLROnPlateau reduz automaticamente a taxa de aprendizagem quando o modelo deixa de apresentar evolução significativa.

Além disso, o ModelCheckpoint salva automaticamente a melhor versão do modelo durante o treinamento, permitindo utilizar posteriormente os pesos que apresentaram melhor desempenho.

4.7 Avaliação do Modelo

Após o treinamento, o modelo foi utilizado para realizar previsões sobre o conjunto de teste.

Os valores previstos foram comparados aos valores reais utilizando três métricas amplamente empregadas em problemas de regressão:

- Mean Squared Error (MSE);
- Root Mean Squared Error (RMSE);
- Mean Absolute Error (MAE).

Essas métricas permitem quantificar o erro médio das previsões realizadas pela rede neural.

Além das métricas numéricas, foram gerados gráficos comparando os valores reais e previstos, possibilitando uma análise visual da qualidade das previsões produzidas pelo modelo.

5. Implementação

O sistema foi desenvolvido utilizando a linguagem Python, adotando os princípios da Programação Orientada a Objetos (POO) e as boas práticas de desenvolvimento propostas pelo Clean Code. A arquitetura foi organizada em módulos independentes, nos quais cada classe possui uma responsabilidade específica, facilitando a manutenção, reutilização e compreensão do código.

A execução do sistema é realizada exclusivamente pelo arquivo `main.py`, responsável por instanciar a classe principal do projeto e iniciar todo o fluxo de processamento.

A organização do projeto é composta pelos seguintes módulos:

- **dados:** responsável pelo carregamento do dataset, normalização dos valores e geração das sequências temporais utilizadas pela rede LSTM.
- **modelo:** responsável pela criação da arquitetura da rede neural, treinamento do modelo e realização das previsões.
- **metricas:** responsável pelo cálculo das métricas de desempenho, incluindo MSE, RMSE e MAE.
- **visualizacao:** responsável pela geração dos gráficos utilizados para análise dos resultados.
- **controlador:** responsável por coordenar todas as etapas da execução do sistema.

Durante a execução, o sistema realiza automaticamente as seguintes etapas:

1. Carregamento do conjunto de dados.
2. Normalização dos valores da série temporal.
3. Geração das sequências de entrada e saída.
4. Divisão entre treinamento e teste.
5. Construção da arquitetura da rede LSTM.
6. Treinamento da rede neural.
7. Geração das previsões.
8. Desnormalização dos valores previstos.
9. Cálculo das métricas de avaliação.
10. Geração dos gráficos.
11. Salvamento do modelo treinado.

Ao término da execução são gerados automaticamente o modelo treinado, as métricas de desempenho e os gráficos utilizados para análise dos resultados.

6. Resultados

Após o treinamento da rede LSTM, foi possível observar que o modelo conseguiu aprender os padrões temporais presentes na série histórica, produzindo previsões para os valores do conjunto de teste.

Para avaliar o desempenho do modelo foram utilizadas três métricas amplamente empregadas em problemas de regressão:

- Mean Squared Error (MSE);
- Root Mean Squared Error (RMSE);
- Mean Absolute Error (MAE).

As métricas obtidas durante a execução do projeto foram:

Métrica	Valor Obtido
MSE	6246,06
RMSE	79,03
MAE	57,35

Além das métricas numéricas, o sistema gerou automaticamente diversos gráficos para facilitar a interpretação dos resultados:

- gráfico da série temporal original;
- gráfico da função de perda (Loss) durante o treinamento;
- gráfico comparando os valores reais e previstos;
- gráfico de dispersão entre valores reais e previstos;
- gráfico de comparação completa entre a série real e a série prevista.

Esses gráficos permitem verificar visualmente o comportamento da rede neural ao longo do treinamento e analisar a proximidade entre os valores previstos e os valores reais.

Durante o treinamento também foi utilizado o mecanismo EarlyStopping, que interrompe automaticamente o processo quando não há melhoria significativa na função de perda, evitando treinamento excessivo e reduzindo o risco de overfitting.

O uso do `ReduceLROnPlateau` possibilitou reduzir automaticamente a taxa de aprendizagem quando o treinamento apresentou estabilização, contribuindo para uma convergência mais estável.

O `ModelCheckpoint` foi utilizado para salvar automaticamente a melhor versão do modelo obtida durante o treinamento, garantindo que as previsões fossem realizadas utilizando os pesos que apresentaram o menor erro de validação.

De maneira geral, os resultados demonstram que a arquitetura LSTM foi capaz de aprender o comportamento da série temporal utilizada neste trabalho e produzir previsões consistentes para os dados avaliados.

7. Conclusão

Neste trabalho foi desenvolvido um sistema para previsão de séries temporais utilizando Redes Neurais Long Short-Term Memory (LSTM), implementado em Python com o auxílio da biblioteca TensorFlow/Keras.

Inicialmente foram estudados os conceitos relacionados às Redes Neurais Artificiais, Redes Neurais Recorrentes e às limitações apresentadas pelas RNN tradicionais. Em seguida, foi apresentada a arquitetura LSTM, destacando o funcionamento de sua célula de memória e das três portas responsáveis pelo controle do fluxo de informações: Forget Gate, Input Gate e Output Gate.

A implementação foi organizada seguindo os princípios da Programação Orientada a Objetos e do Clean Code, permitindo separar cada etapa do processamento em classes específicas responsáveis pelo carregamento dos dados, normalização, geração das sequências temporais, construção da rede neural, treinamento, previsão, avaliação e geração de gráficos.

Os experimentos realizados demonstraram que a rede LSTM foi capaz de aprender padrões presentes na série temporal e produzir previsões para valores futuros utilizando informações históricas como entrada. Além disso, as métricas MSE, RMSE e MAE permitiram avaliar quantitativamente o desempenho do modelo, enquanto os gráficos possibilitaram uma análise visual da qualidade das previsões obtidas.

Durante o desenvolvimento do projeto foi possível compreender a importância do pré-processamento dos dados, da escolha adequada da arquitetura da rede neural e da utilização de técnicas como `EarlyStopping`, `ModelCheckpoint` e `ReduceLROnPlateau` para melhorar o treinamento do modelo.

Como trabalhos futuros, podem ser exploradas arquiteturas mais profundas, modelos Bidirectional LSTM, mecanismos de atenção (Attention Mechanisms), otimização de hiperparâmetros e a utilização de bases de dados maiores e mais complexas, visando melhorar ainda mais a capacidade de previsão do sistema.

Conclui-se, portanto, que as redes LSTM constituem uma solução eficiente para problemas de previsão de séries temporais, sendo amplamente aplicadas em cenários reais que envolvem dados sequenciais, como previsão de demanda, mercado financeiro, consumo de energia, previsão climática e diversas outras aplicações da Inteligência Artificial.

Referências

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. **Deep Learning**. Cambridge: MIT Press, 2016.

HOCHREITER, Sepp; SCHMIDHUBER, Jürgen. Long Short-Term Memory. *Neural Computation*, v. 9, n. 8, p. 1735–1780, 1997.

HAYKIN, Simon. **Neural Networks and Learning Machines**. 3. ed. New York: Pearson, 2009.

GERON, Aurélien. **Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow**. 3. ed. Sebastopol: O'Reilly Media, 2022.

TENSORFLOW. TensorFlow Documentation. Disponível em: <https://www.tensorflow.org/>. Acesso em: 30 jun. 2026.

PEDREGOSA, Fabian et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.